

System Architecture Document

ReqAgent — AI Requirement Intake & Tracker

Author: Steven Saji Paul | Role: Business Systems Analyst

Version: 1.0 | Date: June 2026 | Status: Final

1. Architecture Overview

ReqAgent follows a three-tier architecture: a Streamlit frontend, a Python business logic layer, and a dual-database persistence layer. The AI processing layer sits between business logic and data tiers, acting as an intelligent transformation engine.

PRESENTATION TIER: Streamlit Web Application

Tab 1: Submit | Tab 2: Tracker | Tab 3: Traceability | Tab 4: Analytics

BUSINESS LOGIC TIER:

Form Processor (validation, orchestration)

AI Processor: Groq API – llama-3.3-70b-versatile

DATA TIER:

MongoDB Atlas (NoSQL) Neon Postgres (Relational)

raw_intake collection requirements table

Unstructured text user_stories table

Flexible schema status_audit table

Fast writes conflicts table

2. The Dual-Database Decision

Why MongoDB for Raw Intake

Requirements intake is inherently unstructured. Every stakeholder describes their need differently. Forcing this into a rigid relational schema at intake time would mean designing columns for every possible field upfront, losing flexibility as intake forms evolve, and running schema migrations every time a new field is added. MongoDB's document model stores each submission exactly as received.

Why Neon Postgres for Structured Output

Once the AI processes a requirement, the output is highly structured and predictable. Every requirement has exactly one MoSCoW classification, one priority, and a status that changes over time. This structured data benefits from SQL queries, foreign key relationships, ACID transactions, and joins for the traceability matrix.

System	NoSQL Layer	Relational Layer
E-commerce	Product catalog (varied attributes)	Orders, payments (structured)
Healthcare	Patient notes (free text)	Appointments, billing (structured)
HR Systems	Resumes, cover letters	Employee records, payroll
ReqAgent	Raw requirement intake	User stories, lifecycle, audit

3. Data Flow

Submission Flow

```
User fills form
-> Form validation (all required fields present)
-> insert_raw_intake() -> MongoDB Atlas
-> get_all_titles_and_descriptions() -> fetch existing for conflict check
-> process_requirement() -> Groq API
returns {user_stories, moscow, priority, conflicts}
-> insert_requirement() -> Postgres requirements table
-> insert_user_stories() -> Postgres user_stories table
-> insert_conflict() -> Postgres conflicts table
-> Display results to user
```

Status Update Flow

```
BA selects new status
-> update_status() called with req_id, new_status, old_status
-> UPDATE requirements SET status = new_status
-> INSERT INTO status_audit (req_id, old_status, new_status, timestamp)
```

4. Technology Decisions

Decision	Chosen	Reason
Frontend	Streamlit	Fastest Python-native UI for data apps
AI Model	Groq LLaMA 3.3 70B	Free tier, fastest inference, confirmed working

Decision	Chosen	Reason
NoSQL DB	MongoDB Atlas	Industry standard, generous free tier
Relational DB	Neon Postgres	Serverless Postgres, scales to zero
Visualization	Plotly	Interactive charts, Streamlit native integration

5. Security Architecture

Area	Implementation
API Keys	Stored in .env file, never in source code
.env Protection	Listed in .gitignore, excluded from all commits
Database Connections	SSL enforced on both MongoDB and Neon Postgres
Secret Scanning	GitHub push protection enabled on repo
Credentials Rotation	All credentials rotated after any accidental exposure